

Reconocimiento de Dígitos con MNIST

Autores: Ángel Campos Salido, Steven Rodríguez Santos, Roberto Ramos Trillo, Sophia Requião de Almeida

Asignatura: Aprendizaje Computacional

Escuela: Escuela Técnica Superior de Ingeniería Informática

Universidad: Universidad de Málaga

Fecha: Diciembre 2025



Índice

Capítulo 1: Introducción

- 1.1 Explicación del ejercicio
- 1.2 Preparación de los datos
- 1.3 Modelos usados

Capítulo 2: SVM (Support Vector Machine)

- 2.1 Explicación del modelo
- 2.2 Preparación de los datos
- 2.3 Búsqueda de hiperparámetros
- 2.4 Entrenamiento del modelo final
- 2.5 Evaluación del modelo final

Capítulo 3: Árboles de decisión

- 3.1 Explicación del modelo
- 3.2 Preparación de los datos
- 3.3 Búsqueda de hiperparámetros
- 3.4 Entrenamiento del modelo final
- 3.5 Evaluación de los modelos

Capítulo 4: Random Forest

- 4.1 Explicación del modelo
- 4.2 Preparación de los datos
- 4.3 Entrenamiento del modelo
- 4.4 Resultados: rendimiento del modelo
- 4.5 Análisis de importancia de variables
- 4.6 Modelo final
- 4.7 Conclusión

Capítulo 5: Perceptrón Multicapa

- 5.1 Explicación del modelo
- 5.2 Preparación de los datos
- 5.3 Reducción de dimensionalidad
- 5.4 Búsqueda de hiperparametros
- 5.5 Entrenamiento del modelo
- 5.6 Selección del Modelo Óptimo
- 5.7 Resultados finales

Capítulo 6: Conclusión

- 6.1 Valoración de los modelos
- 6.2 Conclusión final
- 6.3 Referencias

Capítulo 1: Introducción

(realizado por Ángel Campos Salido)

1.1 Explicación del ejercicio

Delante tenemos un problema de **clasificación de imágenes**, el cual resolveremos con técnicas de **Aprendizaje Automático**.

Para ello, disponemos de una **base de datos de MNIST** de 70000 imágenes de **dígitos** (números del 0 al 9) etiquetadas con el número al que corresponde. Cada imagen tiene dimensiones de **28x28** y está en **escala de grises**, por lo que solo hay un valor (intensidad de color) por cada píxel. Ese valor es 0 si el píxel es blanco y 255 si es negro.

Nuestro objetivo es usar estas imágenes para entrenar modelos que puedan clasificar imágenes similares de forma precisa y, de ahí, encontrar el modelo con mejor precisión.

1.2 Preparación de los datos

Antes de usar los modelos, debemos preprocesar los datos de entrada. Para ello debemos seguir los siguientes pasos:

1. **Importar la base de datos.** Esto lo podemos hacer descargando los datasets para importarlos en local o importarlos directamente de Internet usando librerías. Cada uno de nosotros ha usado un método distinto para obtener el dataset.
2. **Normalizar los valores de cada píxel** para que estén en el rango $[0,1]$. Hay algoritmos para los que se requiere previa normalización de los datos, por lo que este paso es necesario. Para ello, dividimos todos los valores de los patrones de X entre 255 (el valor máximo). Esto nos deja con 0 como valor mínimo, el cual representa el blanco y 1 como valor máximo, el cual representa el negro.
3. **Convertir cada patrón** de una matriz de 28×28 a un vector de 784 características. Esto es necesario en caso de que se importen los datos en formato matriz en lugar de en formato vector.
4. **Hacer un submuestreo** para hacer más ligera la selección de modelo. En nuestro caso, hemos usado 1000 patrones para entrenamiento y 250 para test, lo cual acelera sustancialmente el tiempo de entrenamiento y predicción de cada uno de los modelos.

1.3 Modelos usados

Una vez hemos preparado los datos de forma adecuada, podemos proceder a usar varios modelos con el fin de encontrar aquel que mejor se ajuste a nuestras necesidades: en este caso, el modelo con mayor precisión.

Estos modelos son:

1. **SVM** (realizado por Ángel Campos Salido)
2. **Árboles de decisión** (realizado por Steven Rodríguez Santos)
3. **Random Forest** (realizado por Roberto Ramos Trillo)
4. **Perceptrón Multicapa** (realizado por Sophia Requião de Almeida)

Capítulo 2: SVM (Support Vector Machine)

(realizado por Ángel Campos Salido)

2.1 Explicación del modelo

Las máquinas de vector soporte son un conjunto de algoritmos de aprendizaje supervisado que están relacionados con problemas de clasificación y regresión.

Para dos clases, SVM es un modelo que representa los puntos de muestra en el espacio, separando las clases a 2 espacios lo más amplios posibles mediante un hiperplano de separación definido como el vector entre los dos puntos de las dos clases más cercanos, vector al cual se le llama **vector soporte**.

A la forma de predecir, en función del espacio al que pertenezca, pueden ser clasificadas en una u otra clase.

Para SVM multiclase, que es la que vamos a aplicar al tener 10 clases, hay dos formas de resolver el problema:

- Construimos $k(k-1) / 2$ modelos donde k es el número de clases
- Dividimos cada categoría en otras y combinamos todas

Para poder resolver problemas no lineales, debemos usar un kernel que nos permita resolverlos. En este caso, usamos una función de base radial.

2.2 Preparación de los datos

El primer paso es cargar la base de datos usando el comando `read_mnist()`. Al hacer esto, importamos una base de datos de 70000 imágenes etiquetadas. Cada patrón es un vector de 784 elementos, por lo que no necesito usar “reshape”.

El siguiente paso es asignar a cada variable `X_train`, `X_test`, `Y_train`, `Y_test` los valores correspondientes de la base de datos.

Una vez tenemos eso, normalizamos los valores de `X_train` y `X_test` para que estén en el rango de $[0,1]$ y sean compatibles con el modelo.

Por último, hacemos un submuestreo y, de forma aleatoria, seleccionamos 1000 patrones de train y 250 test. A partir de ahí, creamos `X_train_small`, `X_test_small`, `Y_train_small`, `Y_test_small`, los cuales usaremos para entrenar el modelo y para la predicción.

2.3 Búsqueda de hiperparámetros

Para elegir el modelo adecuado, he decidido hacer uso del Tuning, el cual prueba diversos valores de los hiperparámetros a utilizar. En este caso, queremos averiguar los valores más adecuados de:

1. **cost**: Parámetro de regularización que controla el trade-off entre maximizar el margen y minimizar los errores de clasificación.
Cuanto más alto, más penaliza los errores.
Los valores a testear son $[0.1, 1, 10]$
2. **gamma**: Parámetro del kernel radial que define qué tan lejana es la influencia de un solo ejemplo de entrenamiento.
Cuanto más alto, más reducido es el radio de influencia.
Los valores a testear son $[0.001, 0.01, 0.1]$

A partir de aquí, obtenemos los siguientes resultados, siendo la *performance* el valor de error del modelo con mejores parámetros.

```
> print(tuned$best.parameters)
cost gamma
6    10  0.01
> print(tuned$best.performance)
[1] 0.079
```

2.4 Entrenamiento del modelo final

Una vez obtenemos los mejores hiperparámetros, podemos crear un modelo con los datos de entrenamiento y con los hiperparámetros conseguidos.

```
# 4) Entrenamiento de modelo final

svm_model <- svm(
  x = X_train_small,
  y = as.factor(y_train_small),
  kernel = "radial",
  cost = tuned$best.parameters$cost,
  gamma = tuned$best.parameters$gamma
)

summary(svm_model)
```

Esto nos devuelve un modelo svm con las siguientes características:

```
Parameters:
  SVM-Type:  C-classification
SVM-Kernel:  radial
      cost:  10

Number of Support Vectors:  701

( 60 92 55 79 83 53 68 70 82 59 )

Number of Classes:  10
```

2.5 Evaluación del modelo final

Una vez entrenado el modelo final, podemos hacer una predicción usando los patrones de *test*.

Al tener esta predicción podemos crear una matriz de confusión, la cual sirve para mostrar de forma explícita cuándo una clase es confundida con otra.

		Actual									
Predicted		0	1	2	3	4	5	6	7	8	9
0	28	0	1	0	0	0	0	1	0	0	0
1	0	26	0	0	0	0	0	0	0	0	0
2	0	0	25	0	0	0	0	0	1	1	0
3	0	0	1	19	0	0	0	0	0	1	1
4	0	0	1	0	29	0	0	0	0	0	1
5	0	0	0	1	0	25	0	0	1	0	0
6	0	0	0	0	0	0	22	0	0	0	0
7	0	0	0	1	0	0	0	25	0	0	0
8	0	0	0	0	0	0	0	0	19	0	0
9	0	0	0	0	0	0	0	0	0	0	20

Siendo esta la matriz, tenemos que los elementos de la diagonal son aciertos (ejemplo, hay 28 elementos con clase predicha 0 y clase real 0, lo cual cuenta como acierto), mientras que el resto son fallos.

Para calcular el accuracy, hacemos la división entre el número de patrones bien clasificados (los elementos de la diagonal) y el número total de patrones clasificados.

```
> cat("Precisión del modelo SVM:", accuracy, "\n")
Precisión del modelo SVM: 0.952
```

Para esta predicción el modelo nos devuelve un accuracy de un 95,7%, lo cual nos hace ver que, a priori, se trata de un modelo muy adecuado para resolver este problema, al menos con un número reducido de patrones.

Capítulo 3: Árboles de decisión

(realizado por Steven Rodríguez Santos)

3.1 Explicación del modelo

Los árboles de decisión son un tipo de modelo de aprendizaje supervisado que se puede utilizar para clasificación y regresión. Tiene la estructura de un árbol donde

comienza en un nodo raíz y se va ramificando en ramas y nodos internos hasta llegar a las hojas, que representan el resultado final.

Una de las principales ventajas que tienen los árboles de decisión es que son fáciles de entender e interpretar, lo que los convierte en una excelente opción para problemas donde interpretar los modelos es clave.

Otra ventaja de los árboles de decisión es que no requieren una gran preparación de los datos como la normalización y son capaces de manejar tanto datos numéricos como categóricos.

3.2 Preparación de los datos

Los árboles de decisión son invariantes al escalado. Sus decisiones se toman según un umbral, por lo que el intervalo de los atributos en este caso es irrelevante y la normalización a valores $[0, 1]$ no es necesaria.

En este caso, la preparación de los datos simplemente fue suficiente comprobando que no había valores nulos en la tabla.

3.3 Búsqueda de hiperparámetros

Los hiper parámetros buscados en este caso fueron 3:

- **Parámetro de complejidad (CP):** es una medida que representa lo que puede crecer un árbol en tamaño y profundidad. Esto impacta en su capacidad de generalización. Se puede usar para hacer una poda al árbol y hacerlo más simple.
- **Minbucket:** este parámetro indica el número mínimo de observaciones que deben estar presentes en una hoja para que el modelo haga una partición adicional. Si una hoja tiene menos de este número entonces no se divide más. Esto también ayuda a evitar el sobreajuste del modelo.
- **Maxdepth:** controla la profundidad máxima que puede tener el árbol, es decir, el número máximo de divisiones desde la raíz hasta las hojas. Este parámetro también ayuda a prevenir el sobreajuste.

Para encontrar un modelo con unos buenos hiper parámetros primero he definido en R un método para encontrar el CP óptimo (definido en un vector) usando validación cruzada de 8 pliegues. Los valores de CP a probar empiezan en 0.01 y terminan en 0.1 en incrementos de 0.01(10 valores en total).

Una vez definido la validación cruzada junto con los valores CP, hacemos dos bucles anidados donde para cada valor "MS" de un vector de Minbuckets y para

cada valor “MD” de un vector de Maxdepth entrenaremos un modelo usando “MD” y “MS” a la vez que usamos la validación cruzada con los 10 valores de CP.

De esta forma, al salir de los dos bucles, encontraremos la mejor combinación de estos tres hiper parámetros, y nos quedamos con ese modelo.

```
# Para validacion cruzada
control <- trainControl(method="cv", number=8)
# Grid para parametros a probar
grid <- expand.grid(cp = seq(0.01, 0.1, by = 0.01))

mejor_modelo <- NULL
mejor_acc <- 0
iter <- 0
for (ms in c(1,3,5,8)) {
  for (md in c(15, 20, 30)) {
    control_rpart <- rpart.control(minsplit = ms, maxdepth = md)
    modelo <- train(label ~., data=dtrain, method="rpart", trControl=control, tuneGrid=grid, control=control_rpart)
    acc <- calcular_accuracy_caret(modelo, dtest)
    print(paste("Iter:", iter))
    iter <- iter + 1
    if (acc > mejor_acc) {
      mejor_acc <- acc
      mejor_modelo <- modelo
    }
  }
}
```

3.4 Entrenamiento del modelo final

Podemos encontrar un modelo mejor usando el algoritmo adaboost. R tiene una librería donde tiene el algoritmo de entrenamiento ya implementado y llamando a la función nos genera automáticamente el modelo.

```
##### MODELO BOOSTING #####

modelo_boosting <- boosting(label ~., data=dtrain, boos=TRUE, mfinal=50,
                             control = rpart.control(
                               cp = 0.000,
                               maxdepth = 30,
                               minbucket = 1
                             ))
```

3.5 Evaluación de los modelos

A continuación voy a crear las dos matrices de confusiones para ambos modelos y calcular su accuracy.

pred_hp	0	1	2	3	4	5	6	7	8	9
0	13	0	0	0	1	0	0	0	0	1
1	0	31	2	2	0	0	0	0	1	0
2	1	3	8	2	1	2	2	2	2	3
3	0	0	2	9	0	1	1	0	1	0
4	0	0	1	4	22	0	0	0	1	4
5	3	0	0	2	0	12	0	2	1	0
6	0	0	1	0	1	0	12	0	0	0
7	0	0	0	1	2	0	0	17	0	3
8	3	0	0	4	1	3	2	1	27	0

—> Matriz de confusión para el modelo entrenado con grid search. Con un accuracy de 0.68

	0	1	2	3	4	5	6	7	8	9
0	19	0	0	0	0	0	0	2	0	1
1	0	34	0	0	0	0	0	0	1	0
2	0	0	13	1	0	0	0	0	0	0
3	0	0	1	23	0	0	0	0	0	0
4	0	0	0	0	27	0	0	1	1	1
5	0	0	0	1	0	20	0	0	0	0
6	0	0	0	0	0	1	16	0	0	0
7	0	0	0	1	0	0	0	18	0	1
8	1	0	0	0	0	0	1	0	30	1
9	0	0	0	1	3	2	0	1	1	26

—> Matriz de confusión para el modelo entrenado con adaboost. Con un accuracy de 0.904

Se puede ver que el accuracy del modelo usando el algoritmo adaboost es bastante mejor que el hallado usando grid search.

Capítulo 4: Random Forest

(realizado por Roberto Ramos Trillo)

4.1 Explicación del modelo

Random Forest fue seleccionado como uno de los modelos principales del proyecto por varias razones vinculadas a los objetivos de la práctica:

1. **Equilibrio entre rendimiento y coste computacional.**

Aunque no alcanza la precisión de modelos profundos como CNNs, ofrece resultados muy sólidos sin necesidad de GPU ni tiempos largos de entrenamiento.

2. **Robustez ante ruido y variabilidad.**

MNIST contiene trazos irregulares, grosores diferentes y zonas poco informativas. Random Forest maneja bien esta variabilidad al combinar muchos árboles distintos.

3. **Interpretabilidad.**

A diferencia de las redes neuronales, Random Forest permite obtener la **importancia de cada píxel**, lo que contribuye directamente a la reducción de la complejidad sin comprometer el rendimiento.

4. **Buen comportamiento en datos tabulares.**

Aunque MNIST son imágenes, en Kaggle se presentan en un formato tabular

de 784 columnas, ideal para Random Forest.

Por estas razones, Random Forest es un modelo excelente para establecer una **línea base sólida** dentro de la comparación global del proyecto.

4.2 Preparación de los datos

4.2.1 Submuestreo

Para reducir tiempos de entrenamiento y hacer comparables los cuatro modelos del trabajo, se decidió entrenar Random Forest sobre:

- **1000 imágenes para entrenamiento,**
- **250 imágenes para test interno.**

Este subconjunto se obtiene de forma aleatoria desde el dataset completo de MNIST.

4.2.2 Normalización

Aunque Random Forest no requiere estrictamente normalizar, escalar los píxeles a $[0,1]$ evita que ciertas columnas con valores más altos influyan más de lo necesario en las divisiones de los árboles.

Se utilizó:

```
preproc <- preProcess(train_data[ , -1], method = "range")
```

4.3 Entrenamiento del modelo

El modelo se entrenó con los siguientes hiperparámetros:

- **ntree = 300:** número de árboles del bosque.
Esto mejora la estabilidad sin aumentar excesivamente el tiempo de entrenamiento.

- **mtry = 30**: número de píxeles considerados aleatoriamente en cada división. Este valor es cercano a $\sqrt{784} \approx 28$, que es el recomendado teóricamente para clasificación.
- **importance = TRUE**: necesario para obtener la importancia de variables.

Código utilizado:

```
rf_model <- randomForest(
  x = x_train,
  y = y_train,
  ntree = 300,
  mtry = 30,
  importance = TRUE
)
```

4.4 Resultados: rendimiento del modelo

Tras predecir sobre el conjunto interno de 250 ejemplos:

```
conf_mat <- confusionMatrix(pred_test, y_test)
```

Se obtiene una **accuracy aproximada entre 0.89 y 0.92** (depende de la semilla y del subconjunto seleccionado).

Interpretación de los resultados

- Con solo **1000 imágenes**, sin filtros, sin PCA y sin técnicas de visión artificial, el rendimiento es notable.
- La mayoría de errores se dan entre dígitos visualmente similares:
 - **4 ↔ 9**,
 - **3 ↔ 5**,

- 7 ↔ 9.

Comparado con modelos más costosos (p. ej., SVM con kernels complejos o redes neuronales), Random Forest ofrece una relación **precisión / coste computacional** muy atractiva.

4.5 Análisis de importancia de variables

Una de las ventajas clave de este modelo es poder visualizar **qué píxeles son realmente útiles** para el clasificador:

```
varImpPlot(rf_model)
```

Conclusiones del análisis

- Los píxeles del **centro de la imagen** son claramente los más importantes.
- Las esquinas y bordes casi nunca influyen en la predicción.
- Esto valida que se pueden aplicar reducciones de complejidad como:
 - recortes de imagen,
 - eliminación de columnas irrelevantes,
 - PCA enfocada en zonas activas.

Este punto conecta directamente con el objetivo de la práctica de optimizar recursos.

4.6 Modelo final con los 100 primeros dígitos

La práctica exige entregar un modelo entrenado únicamente con los **primeros 100 registros** del dataset original.

Se entrenó un pequeño Random Forest:

```
rf_model_100 <- randomForest(  
  x      = x_train_100,
```

```
y      = train_100$label,  
ntree = 200,  
mtry  = 30  
)
```

Y se guardó con:

```
save(rf_model_100, file = "rf_model_100.RData")
```

4.7 Conclusiones del modelo Random Forest

- Ofrece una **accuracy cercana al 90%** incluso con un dataset reducido.
- Es **rápido, estable y relativamente ligero**, ideal para comparación con otros modelos.
- Permite analizar la **importancia de los píxeles**, facilitando técnicas de reducción de complejidad.
- Actúa como un **modelo de referencia sólido** dentro del conjunto de modelos del proyecto.

Capítulo 5: Perceptrón Multicapa

(realizado por Sophia Requião de Almeida)

5.1 Explicación del modelo

El Perceptrón Multicapa (MLP), fue seleccionado por ser una arquitectura robusta y eficaz para tareas de clasificación en datos estructurados o aplanados, como las imágenes del dataset MNIST.

1. **Complejidad de MNIST:** Clasificar dígitos manuscritos requiere capturar características no lineales, lo cual el MLP maneja bien gracias a sus capas ocultas y funciones de activación no lineales (como ReLU).
2. **Compatibilidad con PCA:** El MLP es ideal para recibir el *input* de características reducidas y aplanadas provenientes del PCA.
3. **Flexibilidad:** Permite la optimización de hiperparámetros cruciales a través del **Grid Search**.

5.2 Preparación de los datos

La preparación es fundamental para asegurar que los datos sean adecuados para el entrenamiento del MLP.

A. Subconjunto de Datos

1. Para la fase de prueba de hiperparámetros, se utilizó un subconjunto reducido para acelerar la experimentación: 1000 imágenes para entrenamiento y 250 imágenes para prueba.

B. Transformación y Normalización

1. **Aplanamiento (*Reshape*):** Las imágenes 28x28 se transformaron en un vector plano de 784 características. Esto es necesario para la capa de entrada del MLP.
2. **Normalización:** Los valores de los píxeles (originalmente en el rango [0, 255]) se dividieron por 255. Esto escala los datos al rango [0, 1], mejorando la estabilidad y velocidad del proceso de entrenamiento.

C. Codificación de Etiquetas (*One-Hot Encoding*)

Las etiquetas de clase (dígitos 0 a 9) se convirtieron a formato vector binario *one-hot*. Esta codificación es obligatoria para trabajar con la función de pérdida `categorical_crossentropy` y la activación de salida `softmax`. Ejemplo: El dígito '3' se convierte en [0, 0, 0, 1, 0, 0, 0, 0, 0, 0].

5.3 Reducción de dimensionalidad

Para optimizar el rendimiento y la eficiencia, se aplicó el Análisis de Componentes Principales (PCA). Esta técnica es fundamental para transformar el espacio de características de alta dimensionalidad (784 píxeles) en un subespacio más compacto, reteniendo la máxima información posible.

1. **Objetivo:** Determinar la cantidad mínima de componentes principales (`n_comp95`) necesarios para explicar el **95% de la varianza total** de los datos originales.
2. **Fundamento:** Este umbral asegura que se preserven las características que definen las diferencias entre los dígitos, al tiempo que se desecha la información redundante o el ruido de baja varianza.

Tras determinar `n_comp95`, la dimensión de entrada fue reducida de 784 características a `n_comp95` características y los datos de entrenamiento y prueba se proyectaron sobre los ejes de los componentes seleccionados, lo que significa que cada nueva característica es una combinación lineal de todos los píxeles originales.

5.4 Búsqueda de hiperparámetros

Se empleó la técnica de Búsqueda en Malla (*Grid Search*) para encontrar la arquitectura óptima del MLP que maximiza la precisión.

A. Hiperparámetros Evaluados

El proceso probó 12 modelos únicos buscando los siguientes hiperparámetros:

- **Número de capas ocultas:** define la profundidad de la Red Neuronal.
 - **Valores probados:** [1, 2, 3]
- **Unidades Iniciales:** Define el tamaño de la primera capa oculta.
 - **Valores probados:** [256, 128]
- **Tasa de Dropout:** técnica de regularización que desactiva un porcentaje de neuronas temporalmente en cada paso de actualización y previene que las neuronas dependan excesivamente de otras
 - **Valores probados:** [0.2, 0.4]

B. Compilación del modelo

Una vez que cada arquitectura de red fue construida, el modelo pasa por la fase de compilación donde se establecerán tres componentes esenciales que guiarán el entrenamiento:

- **Optimizador (`optimizer = 'adam'`):** El optimizador es el algoritmo que ajusta los pesos de la red durante la fase de entrenamiento. El optimizador

Adam fue la mejor opción ya que permitió una convergencia rápida y estable del modelo hacia el mínimo de la función de pérdida.

- **Función de Pérdida (`loss = 'categorical_crossentropy'`):** Esta función mide el error del modelo. Se eligió `categorical_crossentropy` porque es el estándar para problemas de clasificación multiclase con etiquetas codificadas en formato *one-hot* (como es el caso de los dígitos MNIST). El objetivo del optimizador Adam es minimizar continuamente este valor.
- **Métrica (`metrics = c('accuracy')`):** La métrica define cómo se evalúa el rendimiento del modelo de una manera interpretable para el usuario. La Accuracy (Precisión) mide la proporción de predicciones correctas sobre el total de imágenes, siendo la métrica clave para la selección del mejor hiperparámetro.

5.5 Entrenamiento del modelo

La etapa de entrenamiento es donde el modelo aprende de los datos, iterando el proceso de optimización:

- **Datos:** El entrenamiento se ejecuta utilizando el conjunto de datos de entrenamiento con reducción de dimensionalidad (`x_train`, `y_train`).
- **Épocas (`epochs = 25`):** Una época representa un **pase completo** de todos los datos de entrenamiento a través de la red (hacia adelante y hacia atrás). Se estableció en **25 épocas** para cada modelo, lo que permite evaluar el potencial de la arquitectura sin consumir excesivo tiempo, dado que es una prueba de *Grid Search*.
- **Batch Size (`batch_size = 128`):** Define el número de muestras que se procesan antes de que los pesos del modelo sean actualizados. Un *batch size* de 128 equilibra la velocidad de cálculo con la estabilidad de los gradientes.
- **Validation Split (`validation_split = 0.2`):** Un 20% de los datos de entrenamiento se reserva internamente como conjunto de validación. Aunque los resultados finales se evalúan en `x_test`, este *split* ayuda a la red a **monitorear el *overfitting*** durante el entrenamiento de manera interna.

5.6 Selección del Modelo Óptimo

Una vez evaluadas las 12 arquitecturas generadas por el *Grid Search*, los resultados se consolidaron para identificar la combinación de hiperparámetros que ofrecía el mejor equilibrio entre rendimiento y complejidad.

Proceso: Los resultados (Loss y Accuracy) de cada modelo fueron recopilados en una tabla de datos y **ordenados en función de la métrica Accuracy** (Precisión), de mayor a menor y el conjunto de hiperparámetros del modelo que ocupó la primera posición se extrajo y se designó como **best_params** para la fase final del proyecto.

5.7 Resultados finales

El modelo con los mejores hiperparámetros, según la evaluación en el conjunto de prueba interno (x_test), alcanzó los siguientes valores de rendimiento:

Accuracy: 0.916 -> el modelo fue capaz de clasificar correctamente 91.6% de las 250 imágenes de test lo que indica que la red ha aprendido patrones robustos y efectivos para la diferenciación de los dígitos

Loss: 0.3175 -> las predicciones del modelo están muy cerca de la distribución de probabilidad correcta para las clases reales.

Capítulo 6: Conclusión

(realizado por Ángel Campos Salido)

6.1 Valoración de los modelos

Para clasificar los modelos de peor a mejor, vamos a usar el accuracy como medida.

El accuracy de cada modelo para 1000 patrones de entrenamiento y 250 de test que hemos obtenido cada uno es:

- SVM: 95.4%
- Árbol de decisión simple: 68%
- Árbol de decisión con Adaboost: 90.4%
- Random Forest: 89.2%
- Perceptrón Multicapa: 91,6%

Si clasificamos cada modelo por su accuracy, obtenemos lo siguiente:

Árbol de Decisión <<<< Árbol de Decisión Adaboost = Random Forest < Perceptrón Multicapa < SVM

6.2 Conclusión final

Por tanto, podríamos concluir que, de todos los modelos usados y juzgando solamente a partir del accuracy, el SVM con Tuning previo es el mejor modelo ya que tiene un accuracy bastante superior al resto.

Sin embargo, después de investigar, es muy probable que el accuracy del SVM haya salido demasiado alto por un problema de overfitting debido a un “cost” demasiado alto y a que no se ha hecho k-fold cross-validation, por lo que hay que tomar estos resultados con escepticismo.

6.3 Referencias

https://es.wikipedia.org/wiki/M%C3%A1quina_de_vectores_de_soporte

<https://cran.r-project.org/web/packages/e1071/e1071.pdf>

<https://telefonicatech.com/blog/como-interpretar-la-matriz-de-confusion-ejemplo-practico>

<https://cienciadedatos.net/documentos/68-redes-neuronales-r>

<https://cran.r-project.org/web/packages/keras/index.html>

<https://cran.r-project.org/web/packages/caret/index.html>

<https://cran.r-project.org/web/packages/caret/vignettes/caret.html>

<https://www.kaggle.com/c/digit-recognizer>

<https://cran.r-project.org/web/packages/randomForest/randomForest.pdf>

<https://pmc.ncbi.nlm.nih.gov/articles/PMC4264614/>